

Lab Practice 5: Advanced P4 Network Functionality

This laboratory introduces two advanced network mechanisms implemented in the P4 data plane: Explicit Congestion Notification (ECN) and VLAN-based network segmentation with per-VLAN DSCP marking. Both exercises use BMv2 (simple_switch) and the same compilation and deployment workflow from the previous laboratory: compile with p4c-bm2-ss, start the Mininet topology with topo.py, and install forwarding rules using simple_switch_CLI.

The guided exercise (ECN) provides a fully working P4 program that detects queue congestion at egress and signals it by modifying the ECN bits of the IPv4 header without dropping packets. The student activity (VLAN) requires completing a P4 program skeleton that implements IEEE 802.1Q tagging and untagging, VLAN-based forwarding isolation, and per-VLAN DSCP marking. All source files are available in the lab repository.

A key difference between the two exercises is the forwarding model: the ECN exercise uses L3 forwarding with IPv4 longest-prefix match (LPM) and per-hop MAC rewriting, whereas the VLAN exercise returns to L2 forwarding and adds header manipulation — inserting and removing an 802.1Q VLAN tag — as the central programming challenge.

Exercise 1: ECN — Explicit Congestion Notification (Guided)

ECN allows a network device to signal congestion to endpoints without discarding packets. A host that supports ECN sends packets with the ecn field of the IPv4 header set to 1 or 2 (ECN-Capable Transport, ECT). When the egress queue depth at a switch meets or exceeds a defined threshold, the switch marks ecn = 3 (Congestion Experienced, CE) in the packet header, informing the receiver that the path is congested. This exercise implements that marking logic in P4.

The topology consists of two P4 switches and four hosts, two per switch. The link between the switches acts as a bottleneck limited to 0.5 Mbps, which allows congestion to be induced reliably using a background traffic generator.



Figure 1: ECN exercise topology: two P4 switches connected by a 0.5 Mbps bottleneck link.

The host and addressing assignments are:

- s1 (Thrift port 9090): h1 at 10.0.1.1/24 on port 1, h11 at 10.0.1.11/24 on port 2, s2 on port 3.
- s2 (Thrift port 9091): h2 at 10.0.2.2/24 on port 1, h22 at 10.0.2.22/24 on port 2, s1 on port 3.

Because the exercise uses L3 forwarding, each host is configured with a default route pointing to a virtual gateway IP. The switch recognizes packets destined for the remote subnet and rewrites the Ethernet destination MAC at each hop. This configuration is applied automatically by topo.py and does not require manual intervention.

Step 1: Compile the P4 program. From the ecn folder in the lab repository:

```
mkdir -p p4src/build
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json \
--p4runtime-files p4src/build/p4info.txt p4src/ecn.p4
```

Also make the two helper scripts executable before use:

```
chmod +x send.py receive.py
```

Step 2: Start the Mininet topology:

```
sudo python3 mininet/topo.py
```

The topology script starts two BMv2 switches (s1 on Thrift port 9090, s2 on Thrift port 9091), creates four hosts, configures the bottleneck link, and sets the default route and static ARP entries in each host so that L3 traffic can cross subnets. Leave this terminal open.

Step 3: In a second terminal, install the forwarding rules in both switches:

```
simple_switch_CLI --thrift-port 9090 < s1-commands.txt
```

```
simple_switch_CLI --thrift-port 9091 < s2-commands.txt
```

Each command file contains three LPM entries: two /32 entries for the hosts directly connected to that switch, and one /24 entry for the remote subnet forwarded through port 3 (the inter-switch link). The entries follow the syntax:

```
table_add MyIngress.ipv4_lpm MyIngress.ipv4_forward  
<ip>/<prefix> => <next-hop-mac> <port>
```

Step 4: Verify cross-subnet connectivity from the Mininet CLI:

```
mininet> pingall
```

Because the P4 program implements L3 LPM forwarding only, hosts on the same switch and the same subnet cannot reach each other: ARP requests are L2 broadcasts that the LPM table does not process. The pairs h1–h11 (both on s1, subnet 10.0.1.0/24) and h2–h22 (both on s2, subnet 10.0.2.0/24) will therefore always fail. The expected result is 33% drop: 8 of 12 pairs succeed (all cross-switch combinations) and 4 fail (the two same-switch pairs in each direction). This is correct behavior and does not indicate a configuration error. If any cross-switch pair also fails, confirm that the forwarding rules were installed without errors before proceeding.

Step 5: Observe ECN marking. Standard tools such as ping or iperf do not expose individual packet ECN fields or allow sending traffic with a preset ECN value. Two Scapy-based scripts address this. send.py constructs UDP packets with the TOS byte set to 1 (ECN = 01, signaling that the sender supports ECN) and transmits one packet per second for a configurable count. receive.py runs a Scapy sniffer on the destination interface and prints the raw TOS byte of each arriving packet, making the per-packet marking action of the switch directly observable. Open two additional host terminals from the Mininet CLI:

```
mininet> xterm h2 h22
```

In the h2 terminal, start the Scapy receiver that prints the raw tos byte for each arriving UDP packet:

```
h2# python3 ./receive.py
```

In the h22 terminal, start a UDP server to absorb the background flow:

```
h22# iperf -s -u -p 5001
```

From the Mininet CLI, start a high-rate background flow from h11 to h22 to saturate the bottleneck, then immediately send ECN-marked test packets from h1 to h2:

```
mininet> h11 iperf -c 10.0.2.22 -t 15 -u -b 2M -p 5001 &
```

```
mininet> h1 ./send.py 10.0.2.2 "P4 is cool" 20
```

Observe the tos field in the h2 output. Packets sent by send.py carry tos = 1 (ecn = 1, no congestion signal). When the background flow saturates the bottleneck queue beyond the threshold, the switch changes ecn to 3 (CE), making tos = 3. After the iperf flow ends and the queue drains, the field should return to tos = 1.

P4 Pipeline Overview

The P4 program (p4src/ecn.p4) implements IPv4 LPM forwarding with ECN marking in the egress pipeline. The key design element is splitting the 8-bit tos field of the standard IPv4 header into two independently accessible sub-fields:

```
header ipv4_t {  
    // ... other fields ...  
    bit<6>  diffserv; // differentiated services (DSCP)  
    bit<2>  ecn;      // explicit congestion notification  
    // ... other fields ...  
}
```

This separation lets the egress control modify the ECN bits without touching the DSCP bits. The ingress control applies the LPM table, whose ipv4_forward action sets the egress port and rewrites both Ethernet addresses at each hop:

```
action ipv4_forward(macAddr_t dstAddr, egressSpec_t port) {  
    standard_metadata.egress_spec = port;  
    hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
```

```

hdr.ethernet.dstAddr = dstAddr;
hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
}

```

The egress control reads `standard_metadata.enq_qdepth`, the queue depth at the moment the packet was enqueued. If that depth meets or exceeds `ECN_THRESHOLD = 10` and the packet is ECN-capable (`ecn == 1` or `ecn == 2`), the switch calls `mark_ecn()`, which sets `hdr.ipv4.ecn = 3`:

```

const bit<19> ECN_THRESHOLD = 10;

// In MyEgress.apply():
if (hdr.ipv4.ecn == 1 || hdr.ipv4.ecn == 2) {
    if (standard_metadata.enq_qdepth >= ECN_THRESHOLD) {
        mark_ecn(); // sets hdr.ipv4.ecn = 3
    }
}

```

Marking happens in egress — not ingress — because `enq_qdepth` is only populated after the packet has been placed in the output queue, which occurs between the ingress and egress pipelines. The value is therefore unavailable to the ingress control. Because the `ecn` field changes, `MyComputeChecksum` must recalculate the IPv4 checksum listing `diffserv` and `ecn` as separate fields, since they now occupy the byte that was formerly named `tos`. The deparser emits the Ethernet header followed by the IPv4 header.

Activity 1: Exercise 4 — VLAN Segmentation and DSCP Marking

This activity implements IEEE 802.1Q VLAN tagging in the P4 data plane. Two P4 switches divide four hosts into two VLANs. Packets from hosts arrive untagged at the access switch; the switch inserts an 802.1Q VLAN header and forwards the packet across the trunk link to the peer switch. The receiving switch inspects the VLAN header, forwards the packet to the correct host port, and removes the VLAN header before delivery. Packets whose destination MAC and VLAN ID combination has no matching entry are silently discarded in the data plane, enforcing VLAN isolation without any controller intervention.

Additionally, the P4 program writes a DSCP value into the IPv4 header according to the incoming VLAN. This allows downstream devices or monitoring tools to identify the traffic class using the DSCP field even after the VLAN tag has been removed at the destination switch.

The topology consists of two P4 switches and four hosts:

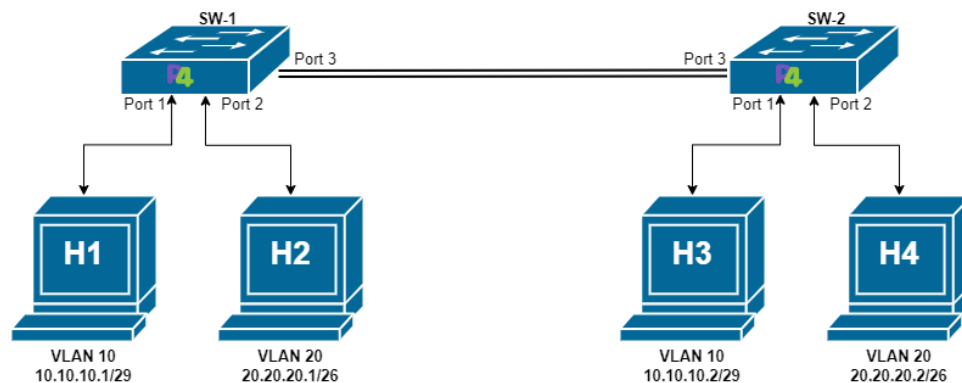


Figure 2: VLAN exercise topology: two access switches connected by a trunk link.

The host and addressing assignments are:

- s1 (Thrift port 9090): h1 at 10.10.10.1/29 (VLAN 10) on port 1, h2 at 20.20.20.1/26 (VLAN 20) on port 2, s2 on port 3 (trunk).
- s2 (Thrift port 9091): h3 at 10.10.10.2/29 (VLAN 10) on port 1, h4 at 20.20.20.2/26 (VLAN 20) on port 2, s1 on port 3 (trunk).

P4 Pipeline Overview

The P4 program (p4src/main.p4) implements bidirectional VLAN processing. The 802.1Q header is defined alongside the standard Ethernet and IPv4 headers:

```
header vlan_802_1q_t {
  bit<3>  pri;          // priority code point (PCP)
  bit<1>  cfi;          // drop eligible indicator (DEI)
  bit<12> vid;          // VLAN identifier
  bit<16> ether_type;  // encapsulated protocol
}
```

Two VLAN identifiers and two DSCP values are defined as constants: VLAN_10 = 0x00a (decimal 10), VLAN_20 = 0x014 (decimal 20), DSCP_VLAN_10 = 10 (AF11), and DSCP_VLAN_20 = 20 (AF22). The parser branches on the Ethernet ether_type: packets with 0x0800 (IPv4) go directly to parse_ipv4; packets with 0x8100 (VLAN TPID) extract the VLAN header first and then continue to parse_ipv4.

The ingress control IngressPipeImpl contains two main tables:

- set_vlan_tag: matched on the source MAC address when the incoming ether_type is 0x0800 (untagged host traffic). The action add_vlan_tag sets the VLAN header valid, assigns the correct VLAN ID, marks the DSCP field of the IPv4 header, moves the original ether_type into the VLAN header, sets hdr.ethernet.ether_type = 0x8100, and forwards the packet to port 3 (trunk).
- extract_vlan_tag: matched on the destination MAC address and VLAN ID when the incoming ether_type is 0x8100 (tagged trunk traffic). The action set_egress_port selects the correct host port. After the table applies, hdr.ethernet.ether_type is restored from the VLAN header and hdr.vlan_802_1q.setInvalid() removes the tag before delivery. The @defaultonly drop default action enforces VLAN isolation: any packet whose destination MAC and VLAN ID combination has no entry is dropped silently.

Because the DSCP field of the IPv4 header is written during tagging, ComputeChecksumImpl must recalculate the IPv4 header checksum after any modification. The deparser emits the Ethernet header, then the VLAN header (only when valid), then the IPv4 header.

TODO Sections to Complete

Open p4src/main.p4 from the VLAN folder in the lab repository and complete all sections marked with TODO. The required changes span the following areas:

- **VLAN ID constants:** open `s1-commands.txt` and locate the `extract_vlan_tag` entries. Each entry includes a decimal VLAN ID as the second match key. Convert those decimal values to 12-bit hexadecimal and assign them to `VLAN_10` and `VLAN_20`. The command files are fixed; the constants you assign must match the keys in those files, so there is only one correct pair of values.
- **DSCP constants:** define two 6-bit values (range 0–63) to assign a traffic class to each VLAN. The expected values are `DSCP_VLAN_10 = 10` (AF11) and `DSCP_VLAN_20 = 20` (AF22). These constants determine the TOS byte visible in packet captures at the destination hosts.
- **Parser (`parse_vlan`):** add a transition `parse_ipv4` statement after `packet.extract(hdr.vlan_802_1q)`, so that IPv4 processing continues correctly when a VLAN header is present.
- **Action `add_vlan_tag` — DSCP marking:** after assigning the VLAN ID, check `hdr.ipv4.isValid()` and write the appropriate DSCP constant to the DSCP bits of the IPv4 header based on the ingress port.
- **Apply block — VLAN removal:** after `extract_vlan_tag` has matched and `hdr.ethernet.ether_type` has been restored, call `hdr.vlan_802_1q.setInvalid()` so the deparser does not emit the VLAN header toward the destination host.
- **Checksum:** add an `update_checksum` call in `ComputeChecksumImpl`, guarded by `hdr.ipv4.isValid()`, covering all IPv4 header fields and storing the result in `hdr.ipv4.hdr_checksum`.

Compile and Deploy

Once the implementation is complete, compile the program from the VLAN folder:

```
mkdir -p p4src/build
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json \
--p4runtime-files p4src/build/p4info.txt p4src/main.p4
```


Start the Mininet topology:

```
sudo python3 mininet/topo.py
```

The topology uses `net.staticArp()` to pre-populate the ARP tables of all hosts after `net.start()`, eliminating ARP resolution as a variable in the VLAN experiment. In a second terminal, install the forwarding rules:

```
simple-switch-CLI --thrift-port 9090 < s1-commands.txt
```

```
simple-switch-CLI --thrift-port 9091 < s2-commands.txt
```

Each command file contains entries for `set_vlan_tag` (one per host connected to that switch, all directing traffic to port 3) and entries for `extract_vlan_tag` (one per host, keyed on destination MAC and VLAN ID in decimal, directing traffic to the corresponding host port). The `extract_vlan_tag` entries follow the format:

Verify VLAN Isolation

From the Mininet CLI, run a full connectivity test:

```
mininet> pingall
```

Observe which pairs succeed and which fail, and relate the result to the VLAN membership of each host. Then test intra-VLAN and cross-VLAN connectivity explicitly:

```
mininet> h1 ping h3 -c 3
```

```
mininet> h2 ping h4 -c 3
```

```
mininet> h1 ping h4 -c 3
```

```
mininet> h2 ping h3 -c 3
```

Hosts h1–h3 (VLAN 10) and h2–h4 (VLAN 20) must communicate successfully. The pairs h1–h4 and h2–h3 must not exchange any packets. The cross-VLAN drop occurs entirely in the P4 data plane through the `@defaultonly` drop action of `extract_vlan_tag`: no packet reaches an unintended host.

Verify DSCP Marking

Open xterm sessions for the destination hosts:

```
mininet> xterm h3 h4
```

In the h3 terminal, capture incoming traffic with verbosity to expose the IP header:

```
h3# tcpdump -i eth0 -v -c 10
```

In the h4 terminal, do the same:

```
h4# tcpdump -i eth0 -v -c 10
```

From the Mininet CLI, generate test traffic while the captures are running:

```
mininet> h1 ping h3 -c 5
```

```
mininet> h2 ping h4 -c 5
```

Inspect the tos field reported by tcpdump. The DSCP value occupies the upper 6 bits of the TOS byte (shifted left by 2), with the two ECN bits set to zero. The expected TOS values are: $\text{DSCP_VLAN_10} = 10 \Rightarrow \text{tos} = 40$ (0x28), and $\text{DSCP_VLAN_20} = 20 \Rightarrow \text{tos} = 80$ (0x50). Verify that each capture shows the value that corresponds to the VLAN of the sender.

Note that the VLAN tag is not visible in the tcpdump output at the host: the switch removes it before delivery. The DSCP marking in the IPv4 header is the only per-VLAN field that survives end-to-end, because it is part of the IP header rather than a link-layer header.

Discussion Questions

1. Which IPv4 header fields are modified in the ECN pipeline and which in the VLAN pipeline? In both cases the IPv4 checksum must be re-computed even though the forwarding goal is different. Explain why checksum recomputation is required in each case.
2. ECN marking occurs in the egress pipeline, whereas DSCP marking in the VLAN exercise occurs in the ingress pipeline. Identify the technical

reason that prevents ECN marking from being performed in ingress, and explain why DSCP marking has no such constraint.

3. What is the fundamental difference between a field that is local to a single link segment, such as an 802.1Q VLAN tag, and a field that is preserved end-to-end, such as DSCP or ECN? How does this distinction explain the switch behavior at access ports (tag insertion and removal) versus the behavior at the trunk port (tag preservation)?